

Algorithm Analysis

Lesson Objectives

By the end of this lesson, students should be able to:

- Understand the concept of algorithm efficiency.
 - Explain time and space complexity.
 - Use **Big-O notation** to describe algorithm performance.
 - Compare and analyze different approaches to solving the same problem.
 - Estimate the number of steps (operation count) for given pseudocode.
-

Introduction: Why Analyze Algorithms?

Two algorithms can solve the same problem but differ in **speed**, **memory use**, and **scalability**.

Example:

- Algorithm A sorts 100 items in **1 second**.
- Algorithm B sorts 100 items in **0.1 seconds**.

If both give the correct output, which is better?

👉 The one that's **more efficient** — faster and less resource-hungry.

Concept of Efficiency

Algorithm efficiency measures how well an algorithm uses **time** and **memory resources** as input size grows.

There are two main types:

1. **Time Efficiency** → How long the algorithm takes.
2. **Space Efficiency** → How much memory it uses.

💡 *Efficiency helps predict how algorithms scale with larger input sizes.*

3 Time Complexity

Definition:

Time complexity describes how the **runtime** of an algorithm increases with input size **n**.

Categories of Time Complexity

Type	Name	Example
$O(1)$	Constant time	Accessing an array element
$O(\log n)$	Logarithmic time	Binary search
$O(n)$	Linear time	Linear search
$O(n \log n)$	Log-linear time	Merge sort, quicksort
$O(n^2)$	Quadratic time	Bubble sort, insertion sort
$O(2^n)$	Exponential time	Recursive Fibonacci
$O(n!)$	Factorial time	Generating all permutations

Example 1: Constant Time — $O(1)$

```
PROCEDURE PrintFirstElement(A)
  PRINT A[0]
ENDPROCEDURE
```

Always takes one operation — constant time.

Example 2: Linear Time — $O(n)$

```
PROCEDURE PrintAllElements(A)
  FOR i ← 0 TO n-1
    PRINT A[i]
  ENDFOR
ENDPROCEDURE
```

If $n = 5$, it prints 5 elements.

If $n = 1000$, it prints 1000 elements.

⇒ Grows linearly with n .

Example 3: Quadratic Time — $O(n^2)$

```
PROCEDURE PrintPairs(A)
  FOR i ← 0 TO n-1
    FOR j ← 0 TO n-1
      PRINT (A[i], A[j])
    ENDFOR
  ENDFOR
ENDPROCEDURE
```

If $n = 10$, total operations = $10 \times 10 = 100$.

Grows quadratically with n^2 .

Space Complexity

Definition:

Space complexity measures how much **memory** an algorithm uses during execution.

Includes:

- Input data size
- Temporary variables
- Recursion stack (if used)

◆ Example:

```
PROCEDURE Sum(A, n)
  total ← 0
  FOR i ← 0 TO n-1
    total ← total + A[i]
  ENDFOR
  RETURN total
ENDPROCEDURE
```

- Only uses variable total and i.
 - Space complexity: **O(1)** (constant).
-

5 Big-O Notation Explained

Big-O gives the **upper bound** on how an algorithm grows with input size.

Formally:

If an algorithm runs in $c \cdot n$ steps (where c is constant), we say it has **O(n)** time complexity.

It helps us compare algorithms **independent of hardware or programming language**.

🏠 Examples of Big-O Simplification

Exact Step Count	Simplified Big-O
$3n + 2$	$O(n)$
$5n^2 + 3n + 8$	$O(n^2)$
$2^n + 5n$	$O(2^n)$

👉 We only keep the **dominant term** as $n \rightarrow \infty$.

6 Comparing Algorithm Approaches

◆ Example: Searching

Algorithm	Method	Time Complexity
Linear Search	Sequential comparison	$O(n)$
Binary Search	Divide list in half each time	$O(\log n)$

✅ Binary search is more efficient — but only works on **sorted data**.

◆ Example: Sorting

Algorithm	Average Time	Space
Bubble Sort	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n)$

7 Visual Example (Diagram Idea)

📊 Graph:

X-axis → Input Size (n)

Y-axis → Execution Time

Lines showing:

- $O(1)$: flat
- $O(\log n)$: slowly rising
- $O(n)$: straight line
- $O(n^2)$: sharply curving upward

(Perfect for slides or printed charts.)

8 Lab: Benchmark Pseudocode Examples

Objective:

Estimate and compare algorithm step counts.

◆ Task 1: Linear Search

Count how many steps are needed for an array of 10, 100, and 1000 items.

◆ Task 2: Nested Loops

Estimate total operations for nested loops of size $n=5$, $n=10$.

◆ Task 3: Compare

Compare total steps for:

- Single loop ($O(n)$)
- Nested loops ($O(n^2)$)

Deliverable:

Write a short report explaining how step counts relate to **Big-O notation**.

9 Git Task

1. Create a new branch named `algorithm_analysis_lab`.
 2. Add your pseudocode and benchmark notes.
 3. Commit and push with:
 4. `git add .`
 5. `git commit -m "Added time and space complexity benchmark exercises"`
 6. `git push origin algorithm_analysis_lab`
-